

Day 3: Embedded Data Types & Keywords

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Fixed-Width Integer Types (stdint.h)

In embedded C++, **never** use `int`, `long`, or `unsigned` without knowing their exact size — these are platform-dependent. Instead, always use:

```
#include <stdint.h>
uint8_t  x;  // exactly 8 bits,  unsigned (0-255)
int8_t   y;  // exactly 8 bits,  signed  (-128-127)
uint16_t z;  // exactly 16 bits, unsigned
int16_t  w;  // exactly 16 bits, signed
uint32_t r;  // exactly 32 bits, unsigned
uint64_t t;  // exactly 64 bits, unsigned
```

On the nRF52840 (32-bit ARM), `int` happens to be 32 bits — but don't rely on this.

volatile Keyword

`volatile` tells the compiler: **"do not cache this variable in a register — always read/write it from actual memory."** Use it for:

- **Hardware registers** (memory-mapped I/O) — the hardware can change the value at any time
- **ISR-shared variables** — the interrupt can modify it between any two instructions in main

```
static volatile bool button_pressed = false;  // Modified in ISR, read in main
volatile uint32_t * const UART_STATUS = (volatile uint32_t *)0x40002108UL;
```

const & constexpr

- `const` — value cannot be changed after initialization. Global `const` lives in Flash (.rodata), saving RAM.
- `constexpr` (C++11) — evaluated at **compile time**. Zero runtime overhead. Use for buffer sizes, timing constants, lookup tables.

```
static const uint8_t MAX_RETRY = 3U;           // Flash (may or may not be inlined)
static constexpr uint32_t BLINK_MS = 500U;    // Guaranteed compile-time constant
```

static in Embedded

`static` has two distinct uses:

1. **File-scope static** — limits symbol visibility to this `.cpp` file (no external linkage). Always use this for module-internal variables and functions — prevents symbol name clashes and reduces binary size.
2. **Local static** — the variable is initialized once and retains its value across function calls (lives in `.bss` / `.data`, not on the stack).

sizeof Guarantees

Use `sizeof` to verify your assumptions about type sizes at compile time:

```
static_assert(sizeof(uint32_t) == 4, "uint32_t must be 4 bytes");
```

This is critical when casting to/from raw byte arrays (e.g., serializing data over UART).

Practice Tasks

1. Use `static_assert` to verify `sizeof(uint64_t) == 8` at compile time
2. Create a `volatile uint32_t` and write/read it — disassemble to see the difference from non-volatile

3. Move a `const uint8_t` array to `constexpr` — check the map file to confirm it stays in Flash
 4. Try using `int` in a struct packed with `uint8_t` — observe the padding with `sizeof`
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day03
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day03/main.cpp](#)